End-to-End Data Management Pipeline for a Recommendation System

Data Management for Machine Learning — Assignment 2

MovieLens latest-small • Python • scikit-learn / SciPy • Apache Airflow

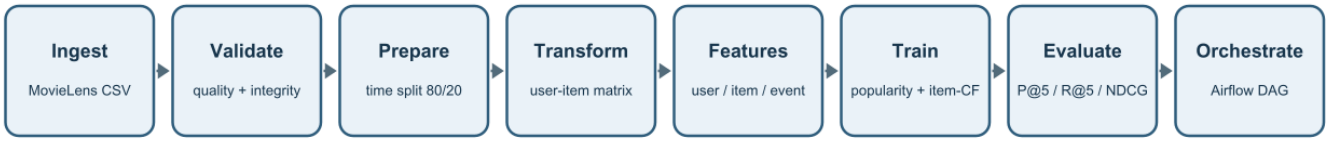
1. Objective and dataset

I implemented a reproducible end-to-end movie recommendation pipeline using the public MovieLens latest-small dataset published by GroupLens Research. The pipeline ingests users, movies and ratings; validates and prepares interactions; creates reusable features; trains two recommenders; evaluates top-K recommendations; and runs through Apache Airflow.

The observed dataset contains 610 anonymized users, 9,742 movies and 100,836 ratings. User demographics are not supplied by this dataset, so the user table contains anonymized identifiers only. Movie titles and pipe-separated genres are used as item metadata. Rating timestamps are converted from Unix seconds to UTC.

Dataset source: https://grouplens.org/datasets/movielens/ • Dataset archive: https://files.grouplens.org/datasets/movielens/ml-latest-small.zip

2. Pipeline architecture



Validated data • train/test sets • feature tables • model artifacts • evaluation report

3. Ingestion and data quality

The ingestion module downloads and extracts the official GroupLens archive on first execution, then normalizes ratings.csv into interactions, movies.csv into item metadata, and anonymized user IDs into a user table. The validator checks required fields, duplicate user-item pairs, rating values in the 0.5–5.0 range with half-star increments, timestamp parsing and user/item referential integrity. Invalid rows are written to a quarantine CSV and aggregate counts are stored as JSON.

Validation check	Invalid rows
Missing required fields	0
Duplicate user-item pairs	0
Invalid ratings	0
Invalid timestamps	0
Unknown users / items	0
Total input / valid / invalid	100,836 / 100,836 / 0

All 100,836 interactions passed the implemented checks. The validation report is included as reports/validation_report.json; the quarantine file is created at data/errors/validation_errors.csv when the pipeline runs.

4. Preparation and feature management

Interactions are sorted chronologically. The earlier 80% is used for training and the latest chronological 20% is held out for testing. The observed split contains 80,669 training rows and 20,167 test rows. The workflow also exports user activity summaries, item popularity summaries, and a user-item rating matrix in CSV and sparse NPZ formats.

Reusable feature tables are generated for users, items and interactions. User features include activity count, average rating, preferred genre and days since last interaction. Item features include rating-count popularity, average rating, title and genres. Interaction features retain rating, UTC timestamp and event type. The feature definitions and availability are documented in reports/feature_catalog.csv.

5. Model training

Popularity baseline: ranks movies by the number of ratings in the training set, excluding movies already seen by the user. Personalized item-item collaborative filtering: builds a sparse user-item rating matrix, computes cosine similarity between item vectors and scores unseen items using each user's observed ratings. Both fitted models are serialized with joblib. Experiment parameters and artifact references are recorded locally; MLflow logging is attempted when the optional dependency is available.

6. Evaluation and comparison

Both models are evaluated at K=5 using Precision@5, Recall@5, NDCG@5, catalog Coverage and genre Diversity. The relevance set is the user's held-out test interactions. Coverage is the fraction of catalog items recommended to at least one evaluated user. Diversity is measured from the genre labels in each recommendation list. RMSE and MAE are not reported because these models rank items and do not predict numeric ratings.

Model	Precision@5	Recall@5	NDCG@5	Coverage	Diversity
Popularity	0.468966	0.036042	0.478816	0.003695	0.563591
Item-item collaborative filtering	0.032759	0.004306	0.031739	0.008417	0.552072

Interpretation. On this run, the popularity baseline outperformed item-item collaborative filtering on Precision@5, Recall@5 and NDCG@5. Collaborative filtering achieved higher catalog coverage (0.008417 versus 0.003695), while genre diversity was similar (0.5521 versus 0.5636). This result is reported as observed; it does not imply that the personalized model is better. The comparison indicates that the current item-CF implementation and evaluation setup would benefit from further tuning and diagnostic analysis.

7. Limitations

MovieLens latest-small is an educational/development dataset and does not establish production performance. User IDs are anonymized and demographic features are unavailable. Cold-start users/items are limited by the interaction history. The project does not implement online serving, drift monitoring, production retraining, privacy controls or a production feature store. The reported metrics are offline top-K results from one chronological holdout split.

8. Orchestration with Apache Airflow

I first verified the end-to-end Python workflow locally with `python -m src.pipeline`. I then installed Apache Airflow separately in WSL2 Ubuntu and created a DAG named recommendation_data_pipeline. Its BashOperator task, run_recommendation_pipeline, executes the existing pipeline using the dedicated recommendation virtual environment and the project directory as its working directory.

Observed local environment

Component	Observed configuration
Operating environment	Windows 11 with WSL2 / Ubuntu
Python in Ubuntu	3.14.4
Apache Airflow	3.4.0
Airflow home / venv	~/airflow / ~/airflow-project/.venv
Recommendation venv	~/recommendation-venv
Airflow UI	http://localhost:8080
DAG / task	recommendation_data_pipeline / run_recommendation_pipeline

Reproducible setup and execution steps

1. In Windows PowerShell, install WSL2 Ubuntu if it is not already available: `wsl --install -d Ubuntu`. Open Ubuntu and install Python tooling with `sudo apt update` and `sudo apt install -y python3 python3-venv python3-pip`.

2. Create and activate a dedicated Airflow environment: `mkdir -p ~/airflow-project && cd ~/airflow-project; python3 -m venv .venv; source .venv/bin/activate`. Install Airflow separately with the matching official constraints file. The captured environment used Airflow 3.4.0.

3. Create a separate recommendation environment: `python3 -m venv ~/recommendation-venv`; activate it and install project dependencies with `pip install -r requirements.txt`.

4. Place `recommendation_pipeline_dag.py` in `~/airflow/dags/`. Configure the DAG's project working directory and Python executable to point to the recommendation project and `~/recommendation-venv/bin/python`. The DAG uses BashOperator to run `python -m src.pipeline` from the project root.

5. Set `AIRFLOW_HOME=~/airflow`, start Airflow with `airflow standalone`, and open `http://localhost:8080`. Log in with the generated admin credentials.

6. In the Airflow UI, locate and enable `recommendation_data_pipeline`, trigger it manually, open Grid view, select `run_recommendation_pipeline`, and inspect Logs. The captured task completed successfully in 13 seconds and exited with return code 0.

The project bundle also includes `reports/AIRFLOW_EXECUTION_GUIDE.md` with detailed commands and configuration notes. Airflow is installed separately rather than through the core `requirements.txt` because its dependencies are version- and platform-constrained.

9. Reproducibility and artifacts

The core pipeline is launched from the repository root using `python -m src.pipeline`. The first execution requires internet access to download the GroupLens archive. The pipeline writes normalized data, train/test splits, summaries, feature tables, sparse matrices, model artifacts, validation JSON, evaluation CSV and run summary. The repository contains modular source code, configuration, requirements, feature catalog, architecture diagram, Airflow DAG, execution guide and captured screenshots.

Repository created for the assignment: github.com/salman253/DM4ML-Assignment-2-Recommendation-Pipeline

Submission note: this report documents the observed successful pipeline and Airflow execution. Before submitting, confirm that the source files, reports and evidence screenshots have been pushed to the public GitHub repository, then provide the repository URL and this PDF to the instructor.

Appendix A — Execution evidence

A1. Successful end-to-end pipeline run

The terminal output shows the real MovieLens dataset counts, 80/20 chronological split, both model metrics and zero invalid validation rows.

```
D:\SALMAN\M.Tech\SEM 2 July 2026 - Dec 2026\Data Management for ML (S1-26_AIMLZG529)\DM4ML Assignment 2\recommendation-pipeline-publi
c-dataset\recommendation-pipeline>python -m src.pipeline
PIPELINE COMPLETED SUCCESSFULLY
{
  "status": "SUCCESS",
  "raw_interactions": 100836,
  "valid_interactions": 100836,
  "train_rows": 80669,
  "test_rows": 20167,
  "models": 2,
  "dataset": "MovieLens latest-small",
  "users": 610,
  "items": 9742
}

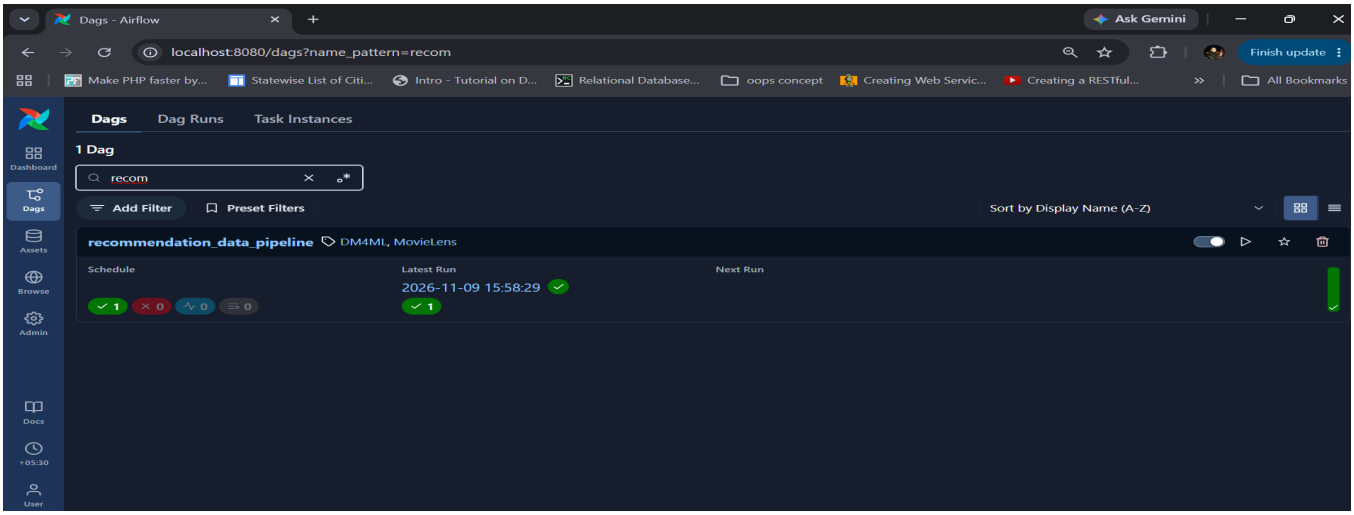
MODEL EVALUATION
                          Model  Precision@5  Recall@5   NDCG@5  Coverage  Diversity
                     Popularity     0.468966  0.036042 0.478816  0.003695   0.563591
Item-item collaborative filtering     0.032759  0.004306 0.031739  0.008417   0.552072

Validation: {'missing_required': 0, 'duplicate_user_item': 0, 'invalid_rating': 0, 'invalid_timestamp': 0, 'unknown_user': 0, 'unknow
n_item': 0, 'input_rows': 100836, 'valid_rows': 100836, 'invalid_rows': 0}

D:\SALMAN\M.Tech\SEM 2 July 2026 - Dec 2026\Data Management for ML (S1-26_AIMLZG529)\DM4ML Assignment 2\recommendation-pipeline-publi
c-dataset\recommendation-pipeline>
```

A2. Airflow DAG registered and successful run

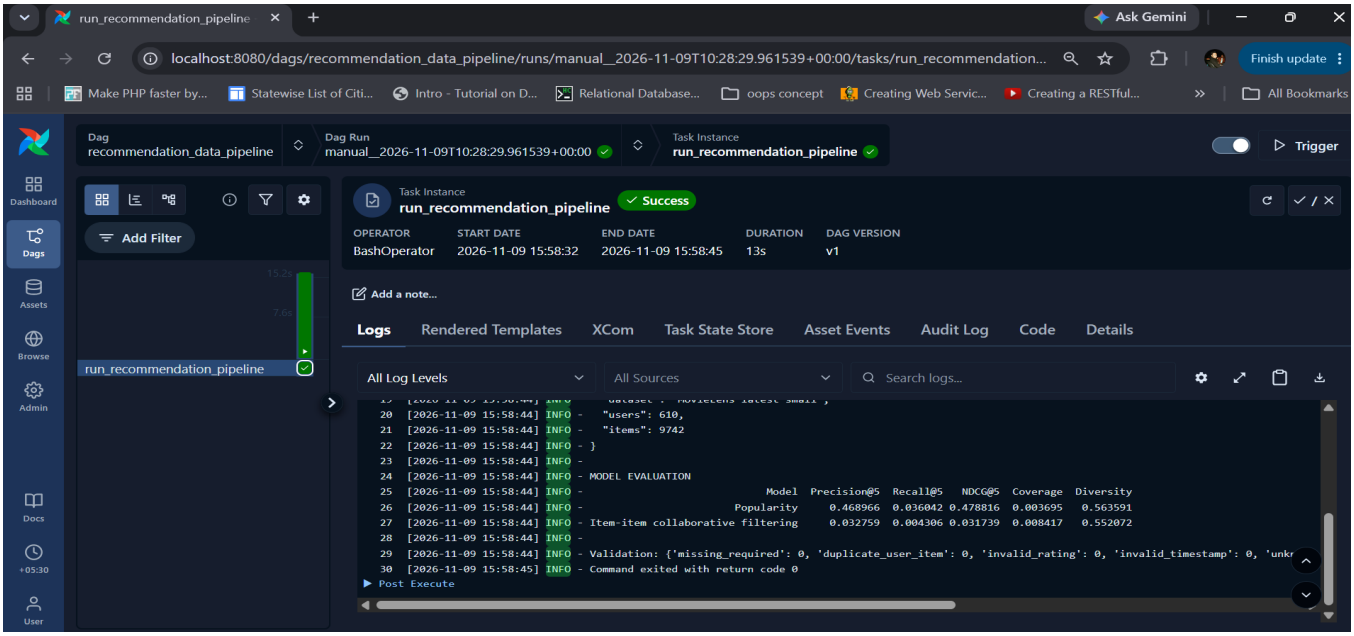
The Airflow DAG list shows recommendation_data_pipeline with one successful run and no failed runs in the captured view.



Appendix B — Airflow task log and references

B1. Airflow task details and logs

The task instance is marked Success. The log displays the MovieLens run summary, evaluation table and validation counts, followed by “Command exited with return code 0”. The task duration shown by Airflow is 13 seconds.



References

1. GroupLens Research. MovieLens datasets. https://grouplens.org/datasets/movielens/ (accessed for this educational assignment).

2. Harper, F. M., & Konstan, J. A. (2015). The MovieLens Datasets: History and Context. ACM Transactions on Interactive Intelligent Systems, 5(4). https://doi.org/10.1145/2827872.

3. Apache Airflow documentation. Installation and DAG authoring documentation. https://airflow.apache.org/docs/ (consult the version-specific documentation when recreating the environment).

Dataset note: MovieLens latest-small is used for educational/development purposes. Review the GroupLens README and terms before reuse; commercial use requires appropriate permission from the dataset provider.